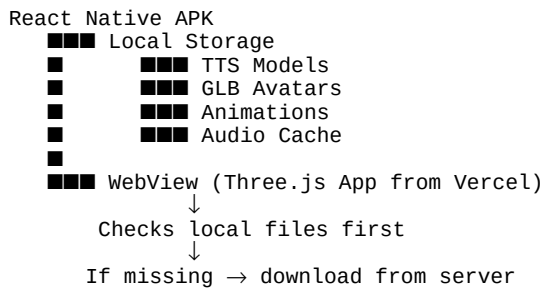


React Native + Three.js Hybrid Offline Architecture

This document explains a scalable hybrid offline-first architecture for integrating a Three.js avatar/TTS system inside a React Native application using WebView, local asset caching, and remote CDN storage.

Architecture Overview



Recommended Flow

1. App launches for the first time.
2. React Native downloads:
 - TTS model
 - Avatar GLB
 - Textures
 - Animations
3. Files are stored locally:
Android:
/data/user/0/com.app/files/

iOS:
Documents/
4. React Native passes local file paths to WebView.

Required Libraries

- react-native-fs
- react-native-webview

Download Example

```
RNFS.downloadFile({
  fromUrl: avatarUrl,
  toFile: localPath,
});
```

Send Local Path to Three.js

```
webViewRef.current.postMessage(JSON.stringify({
  type: "LOAD_AVATAR",
  path: localPath
}));
```

Three.js Message Receiver

```
document.addEventListener("message", async (event) => {
  const data = JSON.parse(event.data);

  if(data.type === "LOAD_AVATAR") {
    loader.load(data.path);
  }
});
```

Fallback Strategy

```
if(local file exists)
  use local
else
  download from CDN/server
```

Fallback Example

```
const exists = await RNFS.exists(localPath);

if(exists){
  useLocal();
}else{
  downloadAndCache();
}
```

Important Security Note

A Vercel-hosted webpage cannot directly scan Android or iOS internal storage because browser sandboxing blocks

Correct Flow:

```
RN App
↓
Gets local file path
↓
Passes path into WebView
↓
Three.js loads file
```

Incorrect Flow:

Vercel app directly scans APK storage

Responsibility Separation

React Native handles:

- File management
- Caching
- Downloads
- Permissions
- TTS model storage

Three.js handles:

- Rendering
- Animation
- Avatar logic
- Lip sync
- Interaction

Recommended Remote Storage

Use CDN/object storage providers:

- Cloudflare R2
- AWS S3
- Firebase Storage

Avoid storing large GLB assets directly in Vercel deployments because:

- Bandwidth costs increase
- Large assets are inefficient on Vercel
- CDN streaming performs better

Production Architecture

React Native APK



WebView



Three.js App

RN handles:

- Model downloads
- Cache
- TTS model storage
- Updates
- Permissions

Three.js handles:

- Rendering
- Animation
- Lip sync
- Interaction

Assets stored on:

- S3 / R2 / Firebase CDN

APK Optimization

Do not package large TTS models directly inside the APK.

Recommended approach:

APK installs



App downloads required models after login



Caches locally

This keeps APK/AAB size manageable for Play Store deployment.

Recommended Asset Sizes

- Avatar GLB: 5-20 MB
- KTX2 compressed textures
- Compressed animation clips
- Lazy-loaded TTS model chunks

Final Recommendation

Recommended stack:

Keep:

- React Native
- WebView
- Three.js webpack app

Add:

- Local caching layer
- RNFS
- WebView bridge communication

Avoid:

- Loading everything from Vercel on every app launch

This architecture is widely used in AI avatar and real-time character applications.